

蜂鸟e203处理器仿真用户程序

- 蜂鸟e203处理器仿真用户程序
 - 版本维护
 - 熟悉E203处理器
 - 任务说明
 - 实现步骤
 - 下载hbird-sdk
 - 配置环境
 - 编译用户程序
 - 运行仿真
 - 修改程序再运行
 - 任务要求

版本维护

版本号	发布日期	说明
v1.0	25/04/06	发布本文档
v2.0	26/04/10	更新nucleisys官网下载中心，完善linux系统环境创建过程

熟悉E203处理器

参考链接：

官方文档给出了使用E203工程进行模拟仿真的流程，有些代码不方便复制的话可以打开这个链接，或者看.md文件而不是看pdf

https://doc.nucleisys.com/hbirdv2/quick_start/simulation.html#

<https://www.jianshu.com/p/89b40e01d983>

环境准备：Ubuntu 18.04/20.04

环境依赖：

```
sudo apt-get install autoconf automake autotools-dev curl device-tree-compiler libmpc-dev libmpc-dev
```

克隆e203工程：

```
git clone https://github.com/riscv-mcu/e203_hbirdv2.git
```

克隆完成之后简单看一下这个工程的结构

- doc：文档，直接看<https://doc.nucleisys.com/hbirdv2/index.html>就可以了
- fpga：FPGA 相关工程
- pics：图片，一般是框图或说明图
- riscv-tools：工具链相关文件
- rtl：最核心的硬件源码
- tb：testbench，仿真测试用
- vsim：仿真脚本和仿真工程

下载GNU编译链到Ubuntu系统中

芯来工具链

Nuclei Studio IDE 2025.10 (win10/11) Windows 10/11 Linux x86-64 User Guide Supply Documents FAQ

Nuclei RISC-V Embedded Toolchain (Baremetal/RTOS + Newlibc) 2025.10 Windows Centos/Ubuntu x86-64 Online Doc

Nuclei OpenOCD 2025.10 Windows 2025.10 2025.02 2024.06 2024.02 2022.12 2022.08 2022.04 2022.01 2020.08 Online Doc

Nuclei QEMU 2025.10 Windows 2024.02 2022.12 2022.08 2022.04 2022.01 2020.08 Online Doc

Nuclei Near Cycle Model 2025.10 Windows 2024.02 2022.12 2022.08 2022.04 2022.01 2020.08 Online Doc

Windows Build Tools 2024.02 Windows 2020.08

Nuclei RISC-V Linux Toolchain (OpenSBI/Uboot/Linux + Glibc) 2025.02-gcc14 Windows Centos/Ubuntu x86-64 Online Doc

然后将编译链链接到e203工程路径下

```
cp nuclei_riscv_newlibc_prebuilt_linux64_2020.08.tar.bz2 ~/
cd ~/
tar -xjvf nuclei_riscv_newlibc_prebuilt_linux64_2020.08.tar.bz2
cd <your_e203_dir>/
mkdir -p ./riscv-tools/prebuilt_tools/prefix/bin

cd ./riscv-tools/prebuilt_tools/prefix/bin/
# 建立riscv gcc的链接
ln -s ~/nuclei_riscv_newlibc_prebuilt_linux64_2020.08.tar.bz2/gcc/bin/* .
```

然后编译自测试样例

在 `riscv-tools/riscv-tests/isa/generated` 目录下重新生成一批可执行测试文件

```
cd <your_e203_dir>/riscv-tools/riscv-tests/isa

source regen.sh
# 测试源码 --> source regen.sh --> generated/ 里的测试可执行文件
```

接下来是运行仿真，运行仿真我们需要用到iverilog仿真工具，以及如果需要查看波形的话，还需要安装gtkwave

但需要注意，不能直接用apt-get安装iverilog（gtkwave可以），因为这样安装的是版本10，而要求的是版本12

```
git clone https://github.com/steveicarus/iverilog.git
# 安装依赖
sudo apt-get install autoconf gperf flex bison build-essential
# 安装
cd iverilog
sh autoconf.sh
./configure
make
sudo make install

# 安装 gtwave
sudo apt-get install gtwave
```

然后编译RTL

```
cd <your_e203_dir>/vsim
make clean
make install
make compile SIM=iverilog
```

编译完成之后跑仿真

运行单个测试

```
make run_test SIM=iverilog
```

如果需要查看波形

```
make wave SIM=iverilog
```

运行测试的工程逻辑如下所示

```
run.makefile 传入 +TESTCASE=测试名
```

↓

```
tb_top 用 $value$plusargs 读到 testcase 路径
```

↓

```
tb_top 用 $readmemh({testcase, ".verilog"}, ...) 读取程序文件
```

↓

```
把程序内容写进 ITCM.mem_r
```

↓

```
释放复位，E203 从 ITCM 取指执行
```

运行回归测试

```
make regress_run SIM=iverilog
```

查看回归测试结果

```
make regress_collect
```

任务说明

前序课程中下载了e203处理器源码、芯来的RISC-V GNU工具链、iverilog仿真工具，能够在e203处理器模型上运行各个指令测试点的仿真，并且得到仿真报告

在前序课程中我们运行的是官方提供的测试程序，本次任务我们要自己编写程序（基于c语言）然后将程序编译后运行在e203处理器上

实现步骤

下载hbird-sdk

在这个网址[GitHub - riscv-mcu/hbird-sdk: OpenSource HummingBird RISC-V Software Development Kit](https://github.com/riscv-mcu/hbird-sdk)下载蜂鸟的sdk工具，SDK（Software Development Kit，软件开发工具包）是一套由软件提供商或其

他组织提供的开发工具，这些工具可以用来创建应用程序或其他软件组件，简言之可以在这个平台上开发能够运行在e203内核上的程序

配置环境

下载好hbird-sdk后，目录如下

```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master$ ls
Build          LICENSE      NMSIS        OS           SoC          doc  npk.yml  package.json  setup.
Components    Makefile    NMSIS_VERSION  README.md  application  setup.bat    test
```

根据github文档中的Requirements和文件结构，我们还需要gcc和openocd来辅助编译

```
File directory:
%NUCLEI_TOOL_ROOT%          //hbird-sdk 根目录
├─ build-tools
│   ├── bin
│   ├── gnu-mcu-eclipse
│   └─ licenses
├─ gcc
│   ├── bin
│   ├── include
│   ├── lib
│   ├── libexec
│   ├── riscv-nuclei-elf
│   └─ share
└─ openocd
    ├── bin
    ├── contrib
    ├── distro-info
    ├── OpenULINK
    ├── scripts
    └─ share
```

```
Requirements:
Ubuntu Linux >=16.04 LTS or Windows >=7
Linux: GNU Make >= 3.82
Windows: Windows Build Tools
Nuclei RISCv GNU GCC Toolchain
Nuclei OpenOCD
```

其中，GCC工具我们在前面已经安装过了，现在只需要将GCC文件夹链接到hbird-sdk文件夹根目录下即可，使用命令

```
ln -s %GCC_ROOT% %NUCLEI_TOOL_ROOT%
```

%GCC_ROOT%表示你的GCC文件夹目录，%NUCLEI_TOOL_ROOT%表示你的hbird-sdk目录，请进行替换

然后是openocd工具，OpenOCD (Open On-Chip Debugger)是一个开源的片上调试器，旨在提供针对嵌入式设备的调试、系统编程和边界扫描功能，也是在芯来的[下载中心](#)直接下载即可



下载完成后得到nuclei-openocd-2025.02-linux-x64.tgz，将其复制到hbird-sdk根目录下，进行解压，运行命令

```
tar -xzf /path/to/nuclei-openocd-2025.02-linux-x64.tgz -C ./
```

这样解压得到的文件夹名字为Nuclei，openocd是里面的子文件夹，所以将其move出来，运行命令

```
mv ./Nuclei/openocd ./
```

现在得到的文件夹目录为

```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master$ ls
Build      LICENSE   NMSIS     OS         SoC        doc  npk.yml  package.json  setup.
Components Makefile  NMSIS_VERSION  README.md  application gcc  openocd  setup.bat    test
```

接下来打开setup.sh文件，将第一行的NUCLEI_TOOL_ROOT修改为自己的hbird-sdk的目录，这是为了配置运行时的环境变量

```
NUCLEI_TOOL_ROOT=~/.hbird-sdk-master
# Create your setup_config.sh
# and define NUCLEI_TOOL_ROOT like below
# NUCLEI_TOOL_ROOT=/home/develop/Software/Nuclei
SETUP_CONFIG=setup_config.sh

[ -f $SETUP_CONFIG ] && source $SETUP_CONFIG

[ -f .ci/build_sdk.sh ] && source .ci/build_sdk.sh
[ -f .ci/build_applications.sh ] && source .ci/build_applications.sh

echo "Setup Nuclei SDK Tool Environment"
echo "NUCLEI_TOOL_ROOT=$NUCLEI_TOOL_ROOT"

export PATH=$NUCLEI_TOOL_ROOT/gcc/bin:$NUCLEI_TOOL_ROOT/openocd/bin:$PATH
```

然后将该文件保存，再在hbird-sdk的目录下运行这个文件

```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master$ source setup.sh
Setup Nuclei SDK Tool Environment
NUCLEI_TOOL_ROOT=/home/wsl_hyh/hbird-sdk-master
```

可以看到现在已经设置好了环境变量，但这个环境变量只在当前terminal有效，每次重启或者新开一个terminal要重新source一下

编译用户程序

cd到这个路径 /hbird-sdk-master/application/baremetal/helloworld

```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master/application/baremetal/helloworld$ ls
Makefile  main.c  npk.yml
```

现在目录下的Makefile描述了编译规则，main.c是源代码，npk.yml是一个配置文件，对编译时的环境进行规定

然后输入命令 `make dasm`


```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master/application/baremetal/helloworld$ make dasm
```

```
Assembling : ../../../../SoC/hbirdv2/Common/Source/GCC/intexc_hbirdv2.S
Assembling : ../../../../SoC/hbirdv2/Common/Source/GCC/startup_hbirdv2.S
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/hbirdv2_gpio.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/hbirdv2_i2c.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/hbirdv2_pwm.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/hbirdv2_spi.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/hbirdv2_uart.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Drivers/htif.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/clock_getres.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/clock_gettime.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/clock_settime.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/close.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/execve.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/exit.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/fork.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/fstat.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/getpid.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/gettimeofday.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/isatty.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/kill.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/link.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/lseek.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/open.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/read.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/sbrk.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/stat.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/times.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/unlink.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/wait.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/Stubs/write.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/hbirdv2_common.c
Compiling  : ../../../../SoC/hbirdv2/Common/Source/system_hbirdv2.c
Compiling  : main.c
Linking    : helloworld.elf
```

text	data	bss	dec	hex	filename
8894	112	2388	11394	2c82	helloworld.elf

```
riscv-nuclei-elf-objdump -S -D helloworld.elf > helloworld.dump
```

```
riscv-nuclei-elf-objdump -d helloworld.elf > helloworld.dasm
```

```
riscv-nuclei-elf-objcopy helloworld.elf -O ihex helloworld.hex
```

```
riscv-nuclei-elf-objcopy helloworld.elf -O verilog helloworld.verilog
```

```
sed -i 's/@800/@000/g' helloworld.verilog
```

编译完成后新的文件结构如下

```
wsl_hyh@DESKTOP-TheNorth:~/hbird-sdk-master/application/baremetal/helloworld$ ls
Makefile          helloworld.dump  helloworld.hex  helloworld.verilog  main.c.o      npk.yml
helloworld.dasm   helloworld.elf   helloworld.map  main.c              main.c.o.d
```

其中，主要关注以下几个文件

helloworld.elf: 通过调试器烧录的可执行文件

helloworld.verilog: 用于运行verilog的输入数据

helloworld.dasm: 反汇编文件，只包含指令和地址

helloworld.dump: 反汇编文件，包含源程序和对照的反汇编指令，更加详细

运行仿真

cd到e203处理器的仿真文件目录中，将生成的 helloworld.verilog 文件拷贝过来，注意更换路径

```
wsl_hyh@DESKTOP-TheNorth:~/e203_hbirdv2-master/riscv-tools/riscv-tests/isa/generated$
cp /home/wsl_hyh/hbird-sdk-master/application/baremetal/helloworld/helloworld.verilog ./
```

再cd到运行仿真目录中，运行仿真命令，请将名字切换成 helloworld

```
wsl_hyh@DESKTOP-TheNorth:~/e203_hbirdv2-master/vsim$ make run_test SIM=iverilog TESTNAME=hellow
```

运行结果如下

```
HummingBird SDK Build Time: Apr  6 2025, 20:43:11
Download Mode: ILM
CPU Frequency 540999 Hz
MISA: 0x40001105
MISA: RV32IMAC
0: Hello World From RISC-V Processor!
1: Hello World From RISC-V Processor!
```

打印出结果之后按 Ctrl+C 退出运行

修改程序再运行

在路径 `~/hbird-sdk-master/application/baremetal` 下，直接将 `helloworld` 文件夹复制一下，将其中的 `Makefile` 文件中的 `TARGET` 修改为自己的工程的名字

```
TARGET = mywork
```

```
HBIRD_SDK_ROOT = ../../..
```

```
SRCDIRS = . src
```

```
INCDIRS = . inc
```

```
COMMON_FLAGS := -O2
```

```
include $(HBIRD_SDK_ROOT)/Build/Makefile.base
```

然后按照前面的步骤复制到e203文件夹中，运行仿真

任务要求

请大家编写程序，然后用e203处理器运行，在终端打印出自己的学号